I Built a Voice Agent That Calls Every New Lead (n8n + Vapi)

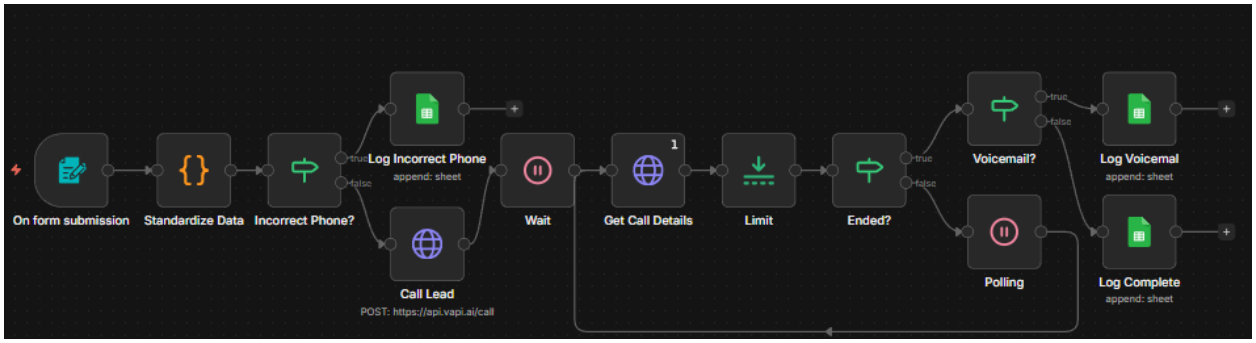
Author: [Nate Herk]

What you're building

A lead qualification voice agent that calls new leads right after they submit a form, asks a few qualifying questions, then logs the results into Google Sheets.

Core flow:

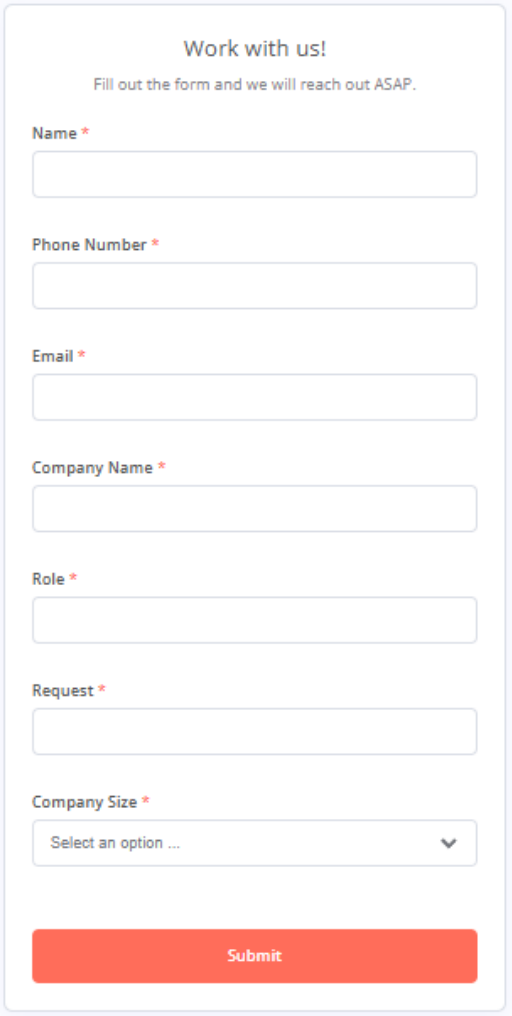
1. Lead submits form (in the demo: n8n Form Trigger).
2. n8n normalizes the phone number.
3. n8n calls the lead using Vapi (Create Call).
4. n8n polls Vapi until the call status is **ended** (Get Call).
5. n8n checks if it was **voicemail**.
6. n8n writes structured outputs (qualification fields) to Google Sheets.



Tools you need

- **n8n** (workflow logic + HTTP requests)
- **Vapi** (voice agent + outbound calling)
- **Google Sheets** (log + pipeline view)
- Optional: **Twilio number** (recommended if you want to scale outbound calls beyond Vapi's daily limits)

Data you collect



1) From the form (initial data)

Minimum fields used in the demo:

- Name
- Phone number
- Email
- Company name
- Role
- Request (what they want help with)
- Company size

2) From the call (extra qualification data)

Pulled from Vapi **Structured Outputs** and logged to the sheet:

- Service interest
- Motivation
- Urgency / timeline
- Past experience
- Budget
- Paid intent (willingness to do paid scoping)
- Status (complete, voicemail, invalid phone, etc.)

High-level wireframe (copy this before you build)

Trigger: Form submission

Step A: Normalize data

- Standardize phone number to a clean 10-digit number.
- If invalid: mark as **incorrect format** and log it.

Step B: Call lead (Vapi)

- Create a call with:

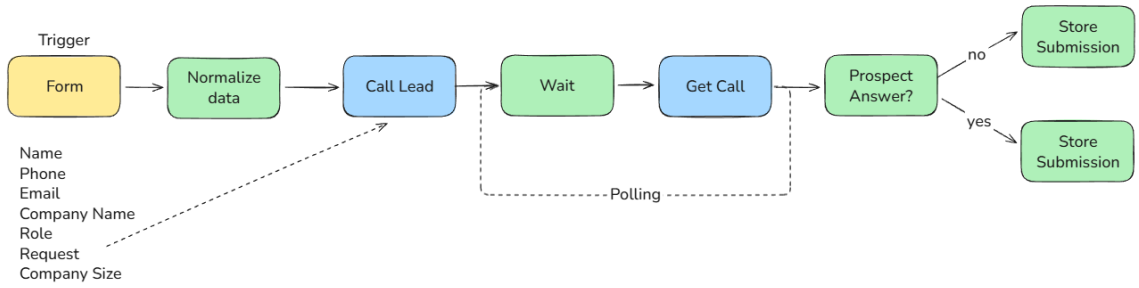
- Assistant ID
- Phone Number ID (caller ID)
- The lead's phone number
- Dynamic variables passed into the assistant prompt

Step C: Poll call status

- Wait (example: 60 seconds)
- Get call details
- If status != ended:
  - Wait (example: 10 seconds)
  - Get call details again
- Repeat until status == ended

Step D: Handle voicemail vs answered

- If endedReason == voicemail:
  - Log voicemail status
- Else:
  - Extract structured outputs and log everything



---

n8n workflow breakdown (node-by-node)

1) Form Trigger (demo)

- Use **n8n Form Trigger** so viewers can replicate with a template.
- In production, this would typically be a **Webhook** receiving data from your website form.

---

2) Code Node: Normalize phone number

Goal: output a 10-digit phone number (no parentheses, dashes, spaces), or output `incorrect format`.

Logic rules used:

- Strip non-numeric characters.
- If number starts with country code like `1`, remove it (US-only assumption).
- If final digit count is not exactly 10, return `incorrect format`.

Tip from the video:

- Copy the incoming JSON from n8n and paste it into Claude (or another assistant) with a one-shot prompt to generate the code node.

---

3) IF Node: Invalid phone?

Condition:

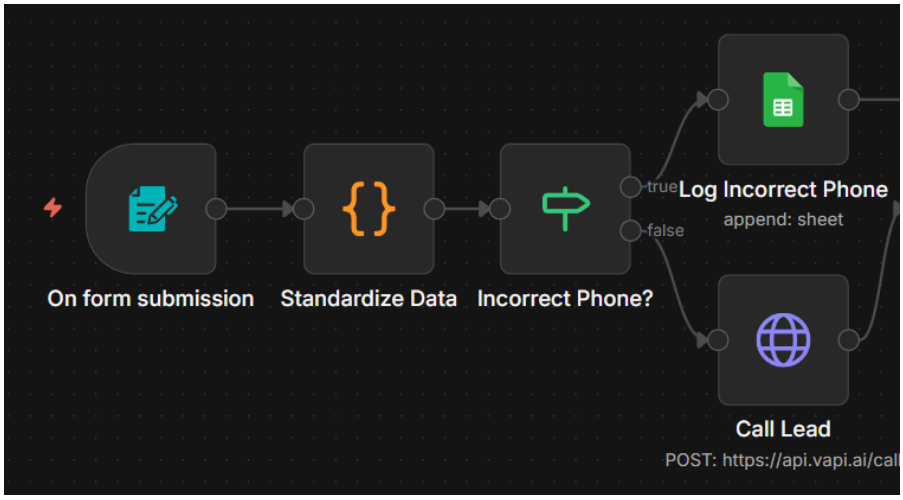
- `phone_number == "incorrect format"`

True branch:

- Log the lead to the sheet with an invalid phone status.

False branch:

- Continue to call lead.



4) HTTP Request: Call Lead (Vapi Create Call)

Purpose: initiates the outbound call.

Method: POST

Auth: Generic Credential, Bearer Auth

- Create a credential in n8n for Vapi.
- Paste your Vapi private API key.

Key body fields (conceptual):

- `assistantId`: your Vapi assistant
- `phoneNumberId`: the Vapi phone number (or imported Twilio number)
- `customer.number`: lead phone number (formatted as `+1` + 10 digits)
- `assistantOverrides.variableValues`: dynamic variables for the assistant prompt

Dynamic variables (the "magic"):
These variables are sent from n8n and injected into the Vapi system prompt:

- `lead_name`
- `lead_company_name`
- `Lead_request`

```
{
  "assistantId": "YOUR ASSISTANT ID",
  "phoneNumberId": "YOUR PHONE NUMBER ID",
  "customers": [
    {
      "number": "+1{{ $json['Phone Number'] }}"
    }
  ],
  "assistantOverrides": {
    "variableValues": {
      "lead_name": "{{ $json.Name }}",
      "lead_company_name": "{{ $json['Company Name'] }}",
      "lead_request": "{{ $json.Request }}"
    }
  }
}
```

---

5) Wait Node (initial wait)

- Example used: **60 seconds**
- This gives time for:
  - The call to ring
  - The conversation to happen

---

6) HTTP Request: Get Call Details (Vapi Get Call)

Purpose: fetch call status and results (including structured outputs).

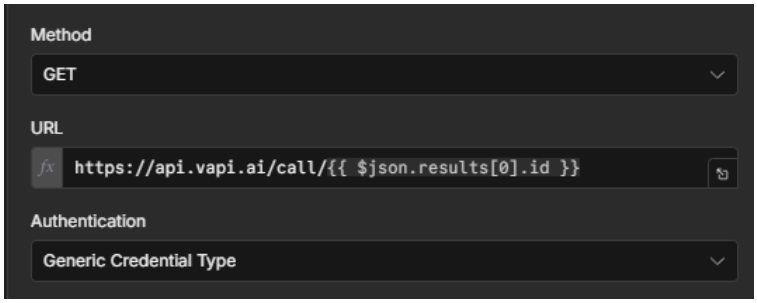
Method: GET

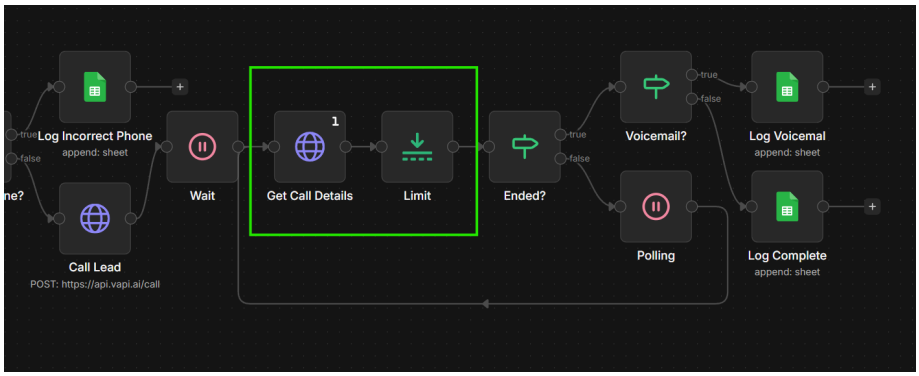
URL pattern:

- The call ID comes from the Create Call response.
- Append it to the endpoint (the docs show a path like `/call/{id}`).

Important:

- In the video, Vapi sometimes returned many items (likely a bug). The workflow limits to the first item as the “source of truth.”





---

7) IF Node: Status ended?

Condition:

- `status == "ended"`

True branch:

- Continue to voicemail check + logging.

False branch:

- Wait (example: **10 seconds**) then loop back to Get Call Details.

This loop is the **polling** system.

---

8) IF Node: Voicemail check

Condition:

- `endedReason == "voicemail"`

True branch (voicemail):

- Log voicemail status to the sheet (so you can call back later).

False branch (answered):

- Extract structured outputs and log full call qualification data.

---

9) Google Sheets: Log results

Create columns for both:

- Form submission fields
- Vapi structured outputs

In the demo, structured outputs map into columns (example: motivation, urgency, budget, intent, etc.).

Vapi setup (assistant)

Model

- Provider: OpenAI
- Model: GPT-4o (as configured in the video)

First message

- Set assistant to **wait for the user to speak first** (more natural when someone answers a call).

System prompt (structure)

Include:

1. **Identity:** who the assistant is (example: “Elliot, outbound lead qualifier for Upit”).
2. **Style + response guidelines:** concise, natural, professional.
3. **Prospect variables:** use double curly braces for dynamic values.
    - `{{lead_name}}`
    - `{{lead_company_name}}`
    - `{{lead_request}}`
4. **Conversation flow + required topics:**
    - Interest confirmation
    - Motivation
    - Urgency / timing
    - Past experience
    - Budget
    - Paid intent (filter tire kickers)
5. **Edge cases:**
    - Wrong number
    - Not a good time
    - User says stop
6. **End call behavior:** enable the built-in “end call” function.

Ethics note used in the video:

- The assistant introduces itself as an AI agent.

Vapi Structured Outputs (how you get clean fields back)

Structured outputs replace older summary/success evaluation setups.

Create structured outputs for the exact fields you want returned, such as:

- Motivation (string)
- Urgency (string)
- Past experience (string)
- Budget (string)
- Service interest (string)
- Paid intent (boolean or string)
- Status (string)

How to configure:

1. In Vapi, open **Structured Outputs**.
2. Add a new one (choose type like string/boolean).
3. Write a clear description of what to extract.
4. Link it to the correct assistant.

Where it appears in the call response:

- Inside a nested object often labeled like **artifacts** → **structuredOutputs**.

Common pitfalls (and fixes)

Phone number formatting causes failed calls

Fix:

- Normalize to a clean 10-digit number.
- Add +1 in the call request if US-only.
- Route invalid numbers to a logging branch.

Call details aren't available immediately

Fix:

- Add polling:
  - Wait 60 seconds
  - Get call
  - If not ended, wait 10 seconds and repeat

Vapi outbound limits on Vapi-purchased numbers

Fix:

- Import your own Twilio number if you need higher outbound volume.

Structured outputs are “hard to find” in response JSON

Fix:

- Drill into the response until you find **artifacts** then **structuredOutputs**.

Testing checklist

- Submit a form with a valid US phone number.
- Confirm the call is initiated.
- Confirm polling loop stops when status becomes ended.
- Test a voicemail scenario and confirm it routes correctly.
- Confirm a row is written to Google Sheets with:
  - Original form data
  - Structured outputs
  - Final status
- Test invalid phone formatting (9 digits, extra digits, country code edge cases).

Quick setup checklist (for viewers)

1. Get your **Vapi API key** and store it as **Bearer Auth** credentials in n8n.
2. Create/configure your **Vapi Assistant** (system prompt + structured outputs).
3. Connect a **phone number** in Vapi (Vapi number or imported Twilio).
4. Update the n8n Call Lead node with:
   - Assistant ID
   - Phone number ID
5. Copy the **Google Sheet template** and connect it to the workflow.
6. Run a test form submission and confirm the sheet populates.

Ideas to extend the workflow

- Add retries if no answer (call again later).
- Send a follow-up SMS if voicemail.
- Auto-create a CRM lead record (HubSpot, Pipedrive, Airtable).
- Auto-book a calendar link if qualified.
- Add call review + prompt tuning loop (analyze transcripts, improve prompt over time).

Want to connect with others building and monetizing AI automation?

Become an AIS Plus Member